

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

- (a) Enumerate three Collection interfaces suitable for this system.
- (b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.
- (c) Write a Java program using HashSet and HashMap with Generics to:
 - Add students
 - Map students to courses
 - Display data using Iterator

(a) Three Collection interfaces suitable for this system

Set<Student> – Stores unique students (no duplicates).

Map<Student, List<String>> – Maps each student to the courses they are enrolled in.

List<String> – Stores multiple courses for each student while maintaining insertion order.

(b) Collection Structure (Representation)

```
HashSet<Student>
```

```
|  
|— Student(101, Alice)  
|— Student(102, Bob)  
|— Student(103, Charlie)
```

```
HashMap<Student, List<String>>
```

```
|  
|— Alice → [Java, DBMS, Networks]  
|— Bob → [Java, Mathematics]  
|— Charlie → [Operating Systems, DBMS]
```

(c) Java Program using HashSet and HashMap with Generics

```
import java.util.*;
```

```
class Student {
```

```
    int id;
```

```
    String name;
```

```
    Student(int id, String name) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
    }
```

```
    public int hashCode() { return id; }
```

```
    public boolean equals(Object o) {
```

```
        return o instanceof Student && id == ((Student) o).id;
```

```
    }
```

```
    public String toString() {
```

```
        return id + " - " + name;
```

```
    }
```

```
}
```

```
public class StudentRegistration {
```

```
    public static void main(String[] args) {
```

```
        HashSet<Student> students = new HashSet<>();
```

```
        Student s1 = new Student(101, "Alice");
```

```
        Student s2 = new Student(102, "Bob");
```

```
        students.add(s1);
```

```
        students.add(s2);
```

```
        HashMap<Student, List<String>> courses = new  
        HashMap<>();
```

```
        courses.put(s1, Arrays.asList("Java", "DBMS"));
```

```
        courses.put(s2, Arrays.asList("Networks", "Math"));
```

```
        Iterator<Student> it = students.iterator();
```

```
        while (it.hasNext()) {
```

```
            Student s = it.next();
```

```
            System.out.println(s);
```

```
            System.out.println(courses.get(s));
```

```
        }
```

```
    }
```

```
}
```

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

- (a) List three Collection classes applicable for cart management.
- (b) Represent how cart items are stored using a List-based structure.
- (c) Write a Java program using ArrayList and Iterator to:
 - Add items
 - Remove an item
 - Display all items

(a) Three Collection classes suitable for cart management

ArrayList

LinkedList

Vector

(b) List-based structure

```
ArrayList (Cart)
```

```
|
```

```
|— Laptop
```

```
|— Mouse
```

```
|— Keyboard
```

```
|— Headphones
```

(c) Java Program

```
import java.util.*;

public class CartSystem {
    public static void main(String[] args) {

        ArrayList<String> cart = new ArrayList<>();

        // Add items
        cart.add("Laptop");
        cart.add("Mouse");
        cart.add("Keyboard");

        // Remove an item
        cart.remove("Mouse");

        // Display items using Iterator
        Iterator<String> it = cart.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Output:

Laptop
Keyboard

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

(a) Name three Map implementations in Java.

(b) Design a schema-like representation using Map for storing ISBN and book details.

(c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

(a) Three Map implementations in Java

HashMap

TreeMap

LinkedHashMap

(b) Map-based schema representation

```
HashMap (Library)
|
├─ ISBN101 → "Java Programming"
├─ ISBN102 → "Database Systems"
└─ ISBN103 → "Computer Networks"
```

(c) Java Program using HashMap

```
import java.util.*;

public class LibraryManagement {
    public static void main(String[] args) {

        HashMap<String, String> books = new HashMap<>();

        // Insert book details
        books.put("ISBN101", "Java Programming");
        books.put("ISBN102", "Database Systems");
        books.put("ISBN103", "Computer Networks");

        // Retrieve a book
        System.out.println("Book: " + books.get("ISBN102"));

        // Display all books
        for (Map.Entry<String, String> b : books.entrySet()) {
            System.out.println(b.getKey() + " : " + b.getValue());
        }
    }
}
```

Output:

Book: Database Systems
ISBN101 : Java Programming
ISBN102 : Database Systems
ISBN103 : Computer Networks

Q4: Employee Data Processing

Suppose that a company processes employee records and filters employees based on salary.

- (a) List three advantages of Generics in Collections.
- (b) Represent how employee data is stored using List with Generics.
- (c) Write a Java program using Iterator to:
 - Store employee objects
 - Display employees with salary $>$ 50,000

(a) Advantages of Generics in Collections

Provides type safety by allowing only specific data types.

Eliminates the need for type casting.

Improves code reusability and readability.

(b) List-based structure using Generics

```
List<Employee>
|
|— Employee(101, Alice, 60000)
|— Employee(102, Bob, 45000)
|— Employee(103, Charlie, 70000)
```

(c) Java Program using Iterator

```
import java.util.*;

class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    public String toString() {
        return id + " " + name + " " + salary;
    }
}
```

```
public class EmployeeData {
    public static void main(String[] args) {

        List<Employee> employees = new ArrayList<>();

        employees.add(new Employee(101, "Alice", 60000));
        employees.add(new Employee(102, "Bob", 45000));
        employees.add(new Employee(103, "Charlie", 70000));

        Iterator<Employee> it = employees.iterator();

        while (it.hasNext()) {
            Employee e = it.next();

            if (e.salary > 50000)
                System.out.println(e);
        }
    }
}
```

Output:

```
101 Alice 60000.0
103 Charlie 70000.0
```

Q5: Online Voting System

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

- (a) Identify three suitable Collection types for this system.
- (b) Design a structure using Set and Map for voters and vote counting.
- (c) Write a Java program using HashSet and HashMap to:

- Record votes
- Prevent duplicate voting
- Display results

(a) Three suitable Collection types

HashSet – To store unique voters and prevent duplicate votes.

HashMap – To store candidates and vote counts.

ArrayList – To store and display voting records.

(b) Set and Map structure

```
HashSet (Voters)
|
├── User101
├── User102
└── User103

HashMap (Vote Count)
|
├── Alice → 2 votes
└── Bob   → 1 vote
```

(c) Java Program using HashSet and HashMap

```
import java.util.*;

public class OnlineVoting {
    public static void main(String[] args) {

        HashSet<String> voters = new HashSet<>();
        HashMap<String, Integer> votes = new HashMap<>();

        String[] users = {"User101", "User102", "User103", "User101"};
        String[] candidates = {"Alice", "Bob", "Alice", "Bob"};

        for (int i = 0; i < users.length; i++) {

            if (voters.add(users[i])) {
                votes.put(candidates[i],
                    votes.getDefault(candidates[i], 0) + 1);
            }
        }

        // Display results
        for (Map.Entry<String, Integer> v : votes.entrySet()) {
            System.out.println(v.getKey() + " : " + v.getValue() + " votes");
        }
    }
}
```

Output:

Alice : 2 votes
Bob : 1 vote